Outbox 与一致性

双写、Transactional Outbox、Saga 与缓存一致性的业务底层

gomall · 业务侧 deck

今日大纲

业务出发点：一致性意味着什么
双写问题：业务上意味着什么
分布式事务全景：六种路线的业务取舍
Transactional Outbox：业务变更 + 事件落表
另一条路线：binlog → MQ (CDC)
DDD 视角：领域事件就是 Outbox 行
Publisher：轮询 + 状态机推进
Publisher 性能调优：批量 / 并发 / 死信处理
事件溯源 + CQRS：把读写模型彻底分离
至少一次语义：下游必须幂等
协同 vs 编排：Saga 的业务取舍
缓存一致性：双删 + 击穿保护
缓存一致性四模式：业务侧选型对比
启动顺序事故：InitLog 必须在 RMQ 之前

一致性是业务事实，不是技术口号

- 业务上的”一致”：库存数、订单状态、搜索结果、用户能看到的页面，四者要对得上。
- gomall 里至少有四个独立存储：MySQL、Redis、RabbitMQ、Elasticsearch。
- 任何一对存储之间都可能”一边成功一边失败”。
- 业务能容忍的不是”零延迟一致”，而是”最终一致 + 看得见的窗口”。

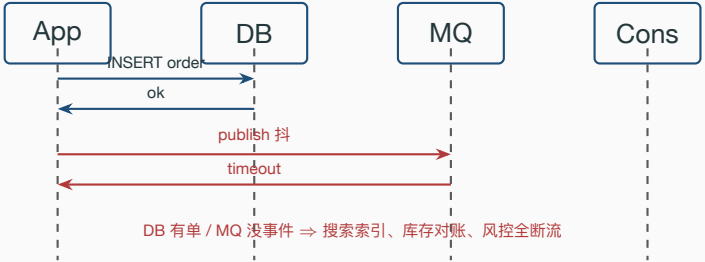
cmd/main.go:29–47 初始化序列里 *4* 个存储依次进场

gomall 里业务侧最在意的三组一致性

一致性对	业务后果	gomall 的兜底	
DB ↔ MQ	下游收不到事件 / 搜索索引腐烂	**Outbox** (deck 主线)	三组
DB ↔ Cache	用户看到旧价 / 旧库存	延迟双删 + SETNX 回源锁	
跨服务步骤	半成品订单、超卖、漏退款	协同式 Saga + 补偿事件	

一致性问题各有典型出错形态。本 deck 只挑这三组，因为 gomall 实际改过、修过、复盘过。*service/order.go*、*service/order_cancel.go*、*service/product.go* 三处对应改动

双写问题的两个不对称失败



反过来：先 publish 再 commit，commit 失败时下游会收到”幽灵订单”，更糟。*docs/blog/04-outbox-saga.md § 这就是双写问题*

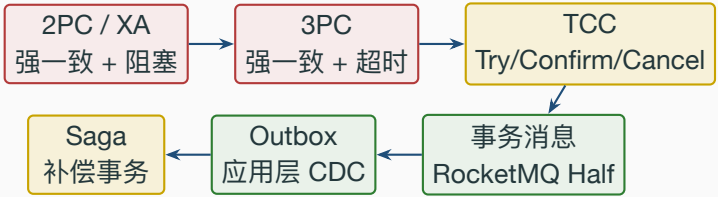
为什么 gomall 不能” 两个 try / catch 套一下”

- DB 事务回滚不会回滚已经发出去的 MQ 消息。
- MQ 投递失败可以重试，但应用进程崩了之后重试上下文也没了。
- 2PC / XA 在 MQ 那一侧支持很差，RabbitMQ 官方没有 XA 资源管理器。
- 业务无法接受” 先停止下单、等运维手动对账”。

业务层结论：必须把” 事件” 也当成 DB 行写进去，让事务统一管。

deck 04 § 双写问题 / 业界标准答案：Transactional Outbox

六种路线一张图：从强一致到最终一致



横轴：左强一致 → 右最终一致；左侧阻塞重、右侧补偿轻。

gomall 选 **Outbox + 协同 Saga**（右下角组合）：单 DB 事务 + 异步事件 + 业务码幂等。对照 *docs/blog/04-outbox-saga.md* 与 *service/outbox/publisher.go*

六种路线参数对比

路线	一致性	锁/阻塞	运维成本	
2PC / XA	强 (ACID)	协调者持锁全程	高 (需 XA broker)	
3PC	强	引入预提交相位	高 (3 阶段超时窗)	
TCC	强 (业务级)	Try 锁定资源	中 (每接口 3 个)	资金/库存
Saga (编排)	最终一致	中心 Coordinator	中 (多一个组件)	
Saga (协同)	最终一致	无中心	低 (事件即 API)	
Outbox	最终一致	单 DB 事务	低 (一张表 + 轮询)	
事务消息 (RocketMQ)	最终一致	Half-Msg 二阶段	中 (RocketMQ 专属)	用 F

一个判断口诀：**资金账户**走 TCC（强一致 + 业务可接受 Try 锁定）；**订单/库存/搜索**走 Outbox + Saga（最终一致 + 业务可接受秒级窗口）。对照 *service/inventory/ (TCC 风格预扣)* 与 *service/outbox/ (最终一致)*

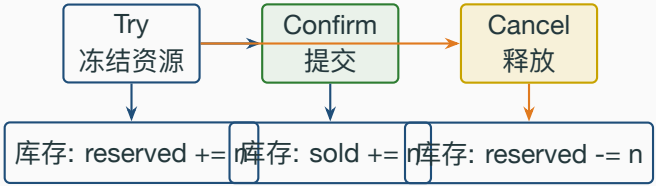
为什么 2PC / XA 在互联网业务里几乎绝迹

- **协调者单点**：协调者宕机时所有参与者都卡在 prepared 阶段持锁。
- **同步阻塞**：prepare 到 commit 之间所有参与者都不能让出资源，p99 被拖死。
- **异构资源支持差**：MySQL 有 XA、Redis 没有、RabbitMQ 没有、ES 没有——四个存储有三个不支持。
- **网络分区时数据被冻结**：见 Jim Gray 论文与 CAP 推论。

> 2PC 强一致的代价是**可用性折半**。gomall 选 AP 路线，把不变量写在业务码里。

docs/blog/04-outbox-saga.md § 为什么不用 2PC

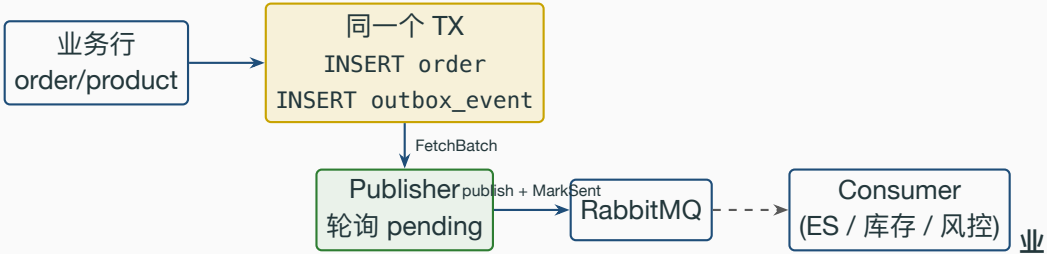
TCC：业务接口翻 3 倍的代价



对照 gomall 实际选型：库存预扣 (reserved) 其实就是 TCC 的 Try 阶段，但 Confirm/Cancel 没走 TCC 协议，而是通过 **Outbox 事件 + 协同 Saga** 异步触发。等价语义、运维成本低一档。

service/inventory/reservation.go (Try=Reserve / Cancel=Release)

Outbox 的核心一句话



业务变更 + 事件写在同一个 DB 事务里——要么都成功，要么都失败。

” 投递给 MQ” 这件事被下沉到一个独立的轮询循环，业务请求不再等 MQ ack。

service/order.go::CreateOrder / service/order_cancel.go::CancelUnpaidOrder

outbox_event 表结构：业务字段就是状态机字段

Listing 1: *repository/db/model/outbox.go:17–37*

```go
type OutboxEvent struct {
    gorm.Model
    AggregateType string // "order" / "product" 业务实体类型
    AggregateID uint // 业务实体 id —— 下游幂等 key 之一
    EventType string // "OrderCreated" / "OrderCancelled"
    RoutingKey string // "order.created" 投递到哪条 routing
    Payload string // JSON body —— 包含一切下游需要的字段
    Status int // 1 pending / 2 sent / 3 dead
    Attempts int // 已尝试投递次数
    NextRetryAt time.Time // 指数退避后的下一次允许时间
    LastError string // 最后一次失败原因，截断 500 字
}
```

几个关键设计：**Status** 是业务码（pending/sent/dead），不是布尔；**NextRetryAt** 让退避不依赖外部调度；**LastError** 留给值班同学复盘。

CancelUnpaidOrder：业务变更 + 事件同事务

Listing 2: *service/order_cancel.go:30–51*

```go
err = baseDao.DB.Transaction(func(tx *gorm.DB) error {
    ok, err := dao.NewOrderDaoByDB(tx).CloseOrderWithCheck(orderNum)
    if err != nil {
        return err
    }
    if !ok {
        return nil // 已经被关过，幂等 no-op
    }
    closed = true
    return dao.NewOutboxDaoByDB(tx).Insert(
        "order", "OrderCancelled", "order.cancelled", order.ID,
        events.OrderCancelled{
            OrderID: order.ID, OrderNum: orderNum,
            UserID: order.UserID, ProductID: order.ProductID,
            Num: order.Num, Reason: "timeout",
        },
    )
})
```

应用层 Outbox vs 数据库 CDC：本质区别

应用层 Outbox
gomall 现状

业务代码 INSERT outbox → Publisher 轮询表 → RMQ

数据库 CDC
binlog → MQ

业务代码只 INSERT order → Canal / Debezium 拉 binlog → Kafk

核心区别：

- Outbox 让业务**显式**声明” 我要发什么事件”，Payload 是业务语义。
- CDC 让业务**完全无感**，事件由 binlog 行级变更倒推出来，Payload 是行差。

对照 *service/order.go* 与假设的 *binlog* 消费链路

Canal / Debezium / Maxwell 三家对比

工具	Canal	Debezium	Maxwell
出身	阿里开源 (Java)	RedHat 开源 (Java)	Zendesk 开源 (Java)
依赖	自带 server / client	Kafka Connect 框架	直接吐到 Kafka/Redis
DDL 支持	解析有限	完整	完整
位点存储	ZK / 本地文件	Kafka topic	MySQL 表 / 文件
适用场景	国内栈、ZK 已有	云原生、K8s、多源	中小项目快速接
gomall 评估	运维有 ZK 才考虑	K8s 成熟后再上	单体阶段过重

三家都装成

MySQL 从库读 binlog，差异在**部署模型**和**下游协议**。gomall 现阶段不引入任何一家——Outbox 一张表 + 一个 goroutine 足够。*docs/blog/04-outbox-saga.md § 业界路线 / 何时该切 CDC*

什么时候 Outbox 该让位给 CDC

切换信号（满足 2 个以上就该评估）：

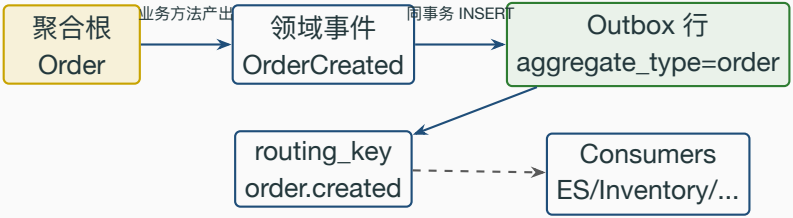
1. 业务表数量 > 20，每张表都要写 outbox 改造成本太高。
2. 跨语言/跨团队消费者，需要”行级变更”这种通用契约。
3. 历史数据回放 / 全量重建索引是常态需求。
4. Publisher 已经横向扩展到 ≥ 4 实例还跟不上写入速度。
5. 离线数仓 / OLAP 也要订阅同一份变更（CDC 天然支持 fanout）。

gomall 现状：表 ≈ 12、消费者全 Go、写入峰值远小于单 publisher 上限——**暂不切换**。

> Outbox 的退场不是被淘汰，而是**规模换路线**。

service/outbox/publisher.go 单实例 *BatchSize=100/1s*，无瓶颈

聚合根 + 领域事件：把业务建模映射到 outbox_event



对照：DDD 的 `AggregateRoot.DomainEvents` 集合，在 gomall 里就是 `tx` 内的若干次 `OutboxDao.Insert`。一次 commit = 一组领域事件落地。*repository/db/model/outbox.go* `AggregateType` / `AggregateID`

业务事件的命名约定：避免事件污染

反例	问题	推荐写法
`OrderChanged`	太泛，下游需要再解析	`OrderCreated` / `OrderCancelled`
`ProductUpdateMsg`	带技术后缀，非业务语义	`ProductPriceChanged`
`UserAction1234`	数字编号，业务不可读	`UserAddressBound`
`TableRowChanged`	表名暴露存储细节	以聚合根 + 业务动作命名

gomall 命名规则（写在 events 包注释）：

- routing_key = `<aggregate>.<verb-past>`，全小写。
- event_type = `<Aggregate><VerbPast>`，PascalCase，与 routing_key 一一对应。
- Payload 字段命名跟随业务术语，不出现 `xxx_id_internal` 这类技术泄露。

service/events/ 包内事件结构体命名

Publisher 默认参数：业务为什么这么取

参数	默认值	业务依据
`PollInterval`	**1s**	用户感知”延迟”上限 1s 是搜索/对账接受的窗口
`BatchSize`	**100**	单实例 + 1 张表 + 长事务规避，100 行单次能在 50ms 内 commit 完
`MaxAttempts`	**10**	指数退避 1s/2s/4s/.../上限 5min，10 次累计可覆盖约 17 分钟波动

这三个值不是拍脑袋：

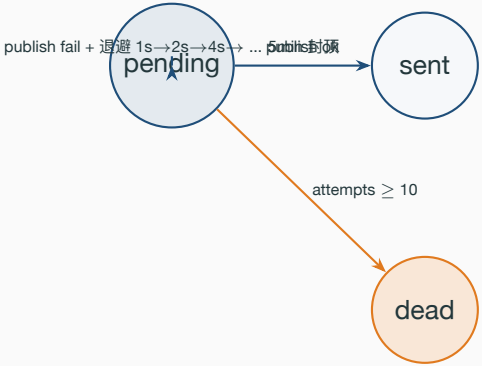
`BatchSize=100` 对应”一次 `FetchBatch` 不会让 publisher 自己变成慢查询”；`MaxAttempts=10` 之后进 dead，由值班看 alarm。*service/outbox/publisher.go:14–32* `withDefaults`；*repository/db/model/outbox.go:14* `OutboxDefaultMaxAttempts`

publisher.loop / drainBatch：主循环 + 状态推进

Listing 3: *service/outbox/publisher.go:51–86*

```go
func (p *Publisher) loop(ctx context.Context) {
   ticker := time.NewTicker(p.cfg.PollInterval); defer ticker.Stop()
   for {
      select {
      case <-ctx.Done(): return
      case <-ticker.C: p.drainBatch(ctx)
      }
   }
}
func (p *Publisher) drainBatch(ctx context.Context) {
   if rabbitmq.GlobalRabbitMQ == nil { return }
   rows, err := dao.NewOutboxDao(ctx).FetchBatch(p.cfg.BatchSize)
   if err != nil { util.LogrusObj.Errorln(err); return }
   for _, ev := range rows {
      if err := rabbitmq.PublishDomainEvent(ctx, ev.RoutingKey, []byte(ev.Payload)); err != nil {
         dao.NewOutboxDao(ctx).MarkFailed(ev.ID, ev.Attempts, p.cfg.MaxAttempts, err.Error())
         continue
      }
      dao.NewOutboxDao(ctx).MarkSent(ev.ID)
```

业务码状态机：pending / sent / dead



- **pending**：默认初值，等下一轮 publisher 取。
- **sent**：业务事件已经送达 broker（注意：不是消费者）。
- **dead**：人工介入分支——值班看 `LastError`，定位下游问题；不会自愈。

repository/db/dao/outbox.go:62–78 `MarkFailed` / *repository/db/model/outbox.go:10–14*

MarkFailed: 指数退避 + 死信落地

Listing 4: *repository/db/dao/outbox.go:62–78*

```go
62 func (d *OutboxDao) MarkFailed(id uint, attempts, maxAttempts int, errMsg string) error {
63    now := time.Now()
64    updates := map[string]any{
65       "attempts": attempts + 1,
66       "last_error": truncate(errMsg, 500),
67       "updated_at": now,
68    }
69    if attempts+1 >= maxAttempts {
70       updates["status"] = model.OutboxStatusDead
71    } else {
72       backoff := time.Duration(1<<attempts) * time.Second
73       if backoff > 5*time.Minute { backoff = 5 * time.Minute }
74       updates["next_retry_at"] = now.Add(backoff)
75    }
76    return d.DB.Model(&model.OutboxEvent{}).Where("id = ?", id).Updates(updates).Error
77 }
```


逐行讲（核心 4 点）：1 << attempts 实现退避 1s/32s/256s，被 5min 封顶；退避结果写

批量 publish：单条 vs 100 条的差距

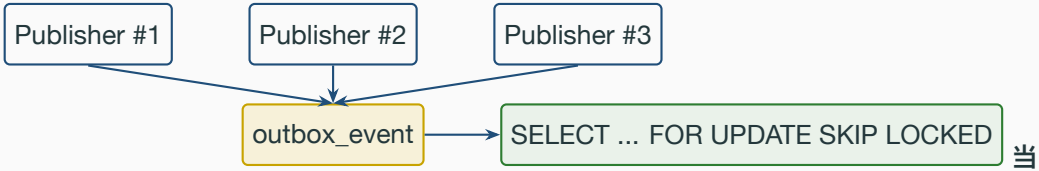
Listing 5: *service/outbox/publisher.go:74–86* 当前单条 *publish* 实现

```go
for _, ev := range rows {
   if err := rabbitmq.PublishDomainEvent(ctx, ev.RoutingKey, []byte(ev.Payload)); err != nil {
      dao.NewOutboxDao(ctx).MarkFailed(ev.ID, ev.Attempts, p.cfg.MaxAttempts, err.Error())
      continue
   }
   if err := dao.NewOutboxDao(ctx).MarkSent(ev.ID); err != nil {
      util.LogrusObj.Errorln("outbox MarkSent:", err)
   }
}
```

当前选型：单条 publish + 单行 UPDATE。

- 优点：失败粒度细——一条挂掉不影响 batch 里其它。
- 缺点：100 条事件 = 100 次 `basic.publish` + 100 次 `UPDATE` = 200 次网络/磁盘 RTT。
- 触发改造：单实例 publish 速度 < 1000 ev/s 时再上批量。

并发 publisher：横向扩展时怎么不抢同一行



前选型：单实例 publisher ——BatchSize=100 / PollInterval=1s 远未到瓶颈。**扩展方案**（写在 SOP 里，触发条件：单实例 backlog $> 10k$）：

- MySQL 8.0+ 用 `SELECT ... FOR UPDATE SKIP LOCKED`，多实例并发取批不重叠。
- 按 `aggregate_id % N` 分片，每个 publisher 只处理一片。
- 加分布式锁（Redis SETNX）做”领导选举”，备份实例只在主挂时接管。

repository/db/dao/outbox.go FetchBatch 后续可加 SKIP LOCKED

死信处理：进 dead 状态之后该做什么

- dead 状态**不会自愈**——这是设计选择：让人介入比让代码盲目重试更安全。
- **监控指标**（Prometheus）：
  - `outbox_pending_total`：堆积量，超过 $10k$ 报警。
  - `outbox_dead_total`：累计死信，每出现一次就报警。
  - `outbox_publish_latency_p99`：单条 publish 延迟，p99 $> 1s$ 报警。
- **人工处置**：值班看 `last_error`，确认下游修复后用 SQL 改回 `status=1, attempts=0, next_retry_at=now()`。
- **反例**：自动把 dead 改回 pending ——等于把” 无限重试” 绕过 MaxAttempts，下游一旦挂掉就被打死。

> 死信信箱的作用是让**失败可见**，不是让**失败被自动忽略**。

repository/db/dao/outbox.go MarkFailed 第 69–70 行 status=Dead 分支

Event Sourcing 与 Outbox 的关系

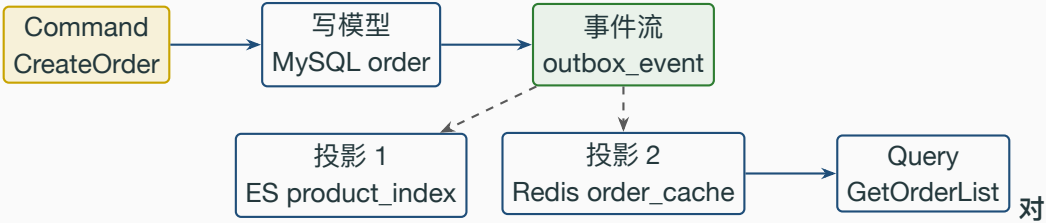
- Outbox：**当前状态**是主存储，事件是副本（用于通知下游）。
- Event Sourcing：**事件流**是主存储，当前状态由事件回放重建。
- gomall 选 Outbox，没选 ES ——但二者技术栈高度重合，可平滑演进。

属性	Outbox (gomall)	Event Sourcing
主真相	`order` 表行	`order_events` 事件流
查询性能	直接读 `order`	必须维护读模型 (projection)
历史可溯源	看 `outbox_event` 表	天然支持
学习/运维成本	低	高 (snapshot / replay)

对照 *Martin Fowler*

eventSourcing.html

CQRS：读模型 / 写模型 / 投影



对

照 gomall 现状：写模型在 MySQL，读模型有两个：ES（搜索）+ Redis（订单列表首页）。两个读模型都靠 outbox 事件维护——**这已经是 CQRS 雏形**，只是没起名字而已。

repository/es/ + repository/cache/order.go 即两个 projection

从 Outbox 到 Event Sourcing 的演进成本

要走到完整 Event Sourcing，至少需要：

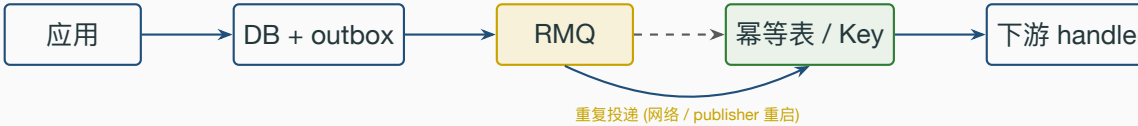
1. 把 `outbox_event` 改成不可删除（当前**已经**没删除路径，只标 sent/dead，符合）。
2. 加 `snapshot` 表：每 *N* 条事件存一次聚合根状态，避免每次查询都从头回放。
3. 写模型从” 先 UPDATE order 再 INSERT event” 反过来——**先 append event，再投影更新 order**。
4. 引入版本号 / optimistic concurrency 控制聚合根并发。

业务决策：gomall 现规模下 **ROI 不正**——改造涉及所有写路径，收益主要是” 完整审计 + 回放调试”。

> Event Sourcing 是 Outbox 的**超集**，不是替代品。

对照 *repository/db/dao/outbox.go (只有 Insert / Mark*，无 Delete)*

Outbox 解决” 丢失”，幂等解决” 重复”



- 投递语义：**at-least-once** ——至少一次。
- 把 **at-least-once + 幂等**做出” 业务侧 exactly-once” 的效果。
- 与 deck 01 / 04 / 05 三个 deck 直接衔接：
  - deck 01 幂等中间件：以 `Idempotency-Key` 兜单次请求。
  - deck 04 outbox：本 deck 的技术细节版本。
  - deck 05 限流熔断：客户端重试时再叠加 RateLimit / CircuitBreaker。

压测里量出来的两组真实数字

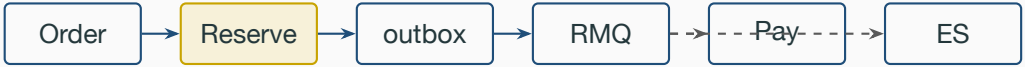
压测场景	
单元测试 `TestCoupon_AtomicClaimNoOversell`：500 并发抢 100 张券	**零**
压测 `idempotency_replay.js`：同 Key 累计 15s 请求	**75**
对应入库订单数（`SELECT COUNT(*)`）	
回放 p95 / 吞吐	2.33ms / **50,319**

”零超发” + ”75 万次只入 1 单” 是同一套设计的两个表现——**at-least-once 投递**遇上**业务**

码幂等键（券号 / Idempotency-Key），最终对账数永远只算一次。*stressTest/REPORT.md § 单元测试一览 & §4. 幂等中间件压测*

协同 vs 编排：哪种 Saga 配 gomall

模式	实现	业务取舍
编排 (Orchestration)	中心 Coordinator 推	步骤 ≥ 5、需要看板、跨团队
协同 (Choreography)	事件驱动、无中心	**gomall 选用**——步骤少

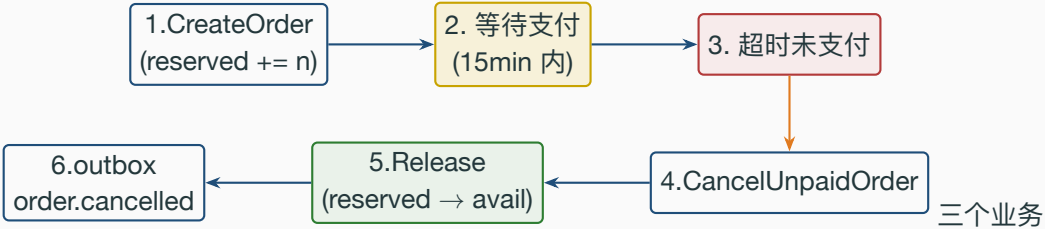


协同式：每一步消费上一步事件、产出下一步事件，没有” 总指挥”。

对 gomall 的 5 步链路 (Order → Reserve → outbox → Pay / ES) 是最省心的选择。

docs/blog/04-outbox-saga.md § 完整链路

Saga 失败一步：CancelUnpaidOrder 就是一次回滚



保证：

- **幂等**：第二次调用 `CloseOrderWithCheck` 返回 `ok=false`，跳过释放与事件。
- **先 DB tx 后 cache**：DB 关单 + outbox 一致后再释放 Redis 占用——失败可重放。
- **最终一致**：库存 = available + reserved + sold 这条不变量始终成立。

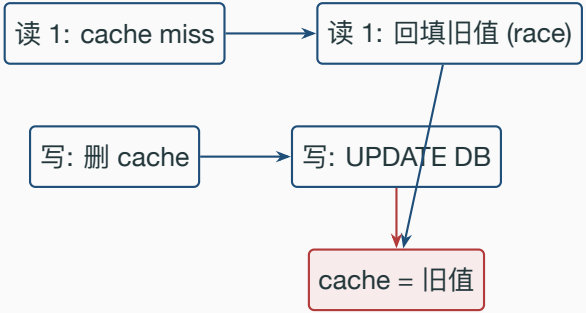
service/order_cancel.go:14–63 完整闭包

业务接受怎样的” 补偿失败”

- Saga 补偿不是银弹：补偿本身也会失败（比如 cache 拒连）。
- gomall 的容错思路：
  1. 先保证 DB 这一边 commit ——业务真相先落地。
  2. Cache 释放失败只打 error 日志，不回滚 DB ——状态机靠下次扫描自愈。
  3. 真出现**长期不一致**时，由 cron + outbox 死信表兜底，值班对账。
- 业务话术（写在 SOP 里）：**” 先关单 + 后释放”，释放失败 = 库存短暂少一点，绝不会卖超。**

service/order_cancel.go:58–62 释放 *reservation* 仅 *log*；*initialize/cron.go* 巡检任务

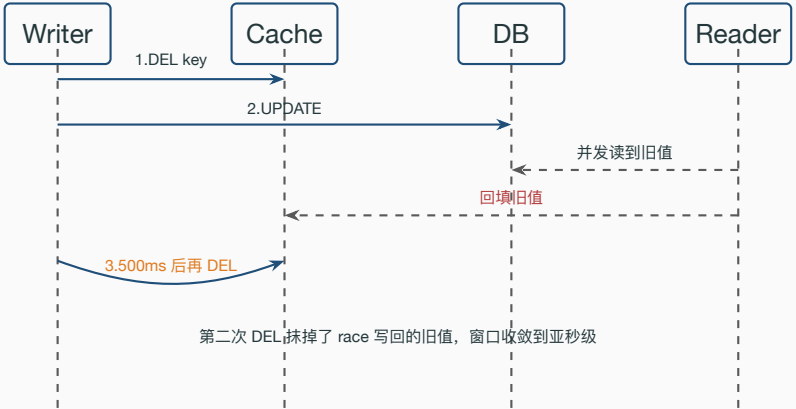
Cache Aside 的两个出错位置



业务出错形态：商品改价后，部分用户在 1～2 秒内还能看到旧价。

对 gomall：**允许**短时间旧库存（最坏拒单），**不允许**长时间旧价格（结算口径不一致）。*service/product.go:282–288 ProductUpdate* 写库前后两次 `DelProductDetail`

延迟双删时序图



repository/cache/product.go:66–74 `DoubleDeleteAsync`

双删 + 击穿保护：源码与逐行讲

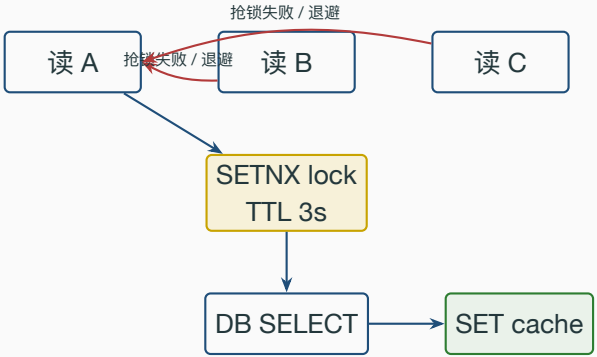
Listing 6: *repository/cache/product.go:55–74*

```go
// TryProductLock SETNX 抢回源锁，避免缓存击穿时多个请求同时回源
func TryProductLock(ctx context.Context, id uint) (bool, error) {
   return RedisClient.SetNX(ctx, ProductLockKey(id), "1", ProductLockTTL).Result()
}

func UnlockProduct(ctx context.Context, id uint) {
   RedisClient.Del(ctx, ProductLockKey(id))
}

// DoubleDeleteAsync 延迟双删：写库后已经做了第一次删除，这里在 interval 后再删一次，
// 用于覆盖"读旧值的并发请求把旧值塞回缓存"的窗口。
func DoubleDeleteAsync(id uint, interval time.Duration) {
   if interval <= 0 { interval = ProductDelayInterval } // 默认 500ms
   go func() {
      time.Sleep(interval)
      _ = RedisClient.Del(context.Background(), ProductDetailKey(id)).Err()
   }()
}
```

击穿保护：SETNX 单飞回源



业务收益：热点商品（首页 banner 商品）失效瞬间不会让 1000 个请求一起打到 DB。

落库次数从 *N* 个请求降到 1 次回源 + 其余请求复用 cache。*repository/cache/product.go:55–62*

`TryProductLock` / `UnlockProduct`

Cache Aside / Write Through / Write Back / Read Through

模式	写路径	读路径	典型问题
Cache Aside	业务写 DB → 删 cache	cache miss → 读 DB → 回填	双删窗口 (go-mall)
Write Through	业务写 cache → cache 同步写 DB	直接读 cache	cache 即真相，故障重
Write Back	业务写 cache，cache 异步刷 DB	直接读 cache	cache 宕机丢数据
Read Through	业务只跟 cache 说话；miss 时 cache 自己回源	同左	需要 cache 中间层

gomall

选 Cache Aside：DB 是真相，cache 失败不影响业务正确性。Write Back 虽快（写 cache 1ms / 异步落 DB），但 Redis 宕机直接丢数据、对账困难、冷启动慢——订单/库存**绝不**走 Write Back；只有”商品浏览次数”这种允许 1% 丢失的副表才用。

真实事故：`tryInitRabbitMQ` 让进程整个退出

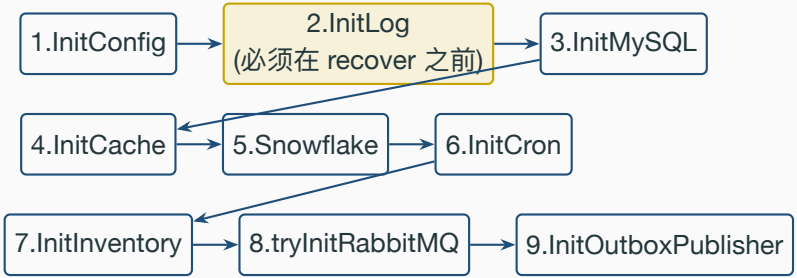
现象：RMQ 没起，`tryInitRabbitMQ()` recover 时 **NPE**，进程 crash。

定位过程：

1. PR #51（订单延迟关单）引入 `tryInitRabbitMQ()`。
2. 该函数 `recover()` 内调用 `util.LogrusObj.Warnf(...)`。
3. 当时 `util.InitLog()` 写在 `tryInitRabbitMQ()` **之后**。
4. `LogrusObj == nil` → recover 内再 panic → 进程退出。

修复（commit 8d0157a / 3cbe3c4）：把 `util.InitLog()` 提前到所有可能 `recover` 的代码之前。*cmd/main.go:30–31 修复后的顺序；stressTest/REPORT.md § 已修：util.LogrusObj*

修复后的启动顺序（业务因果）



关键约束：**InitLog 必须在 InitMySQL / RMQ 之前**，因为 MySQL / RMQ 初始化的 `recover` 都依赖 `LogrusObj`。

cmd/main.go:29–47 完整初始化序列

两条可复用的业务侧教训

1. **初始化顺序**：所有依赖全局对象的代码，必须在它初始化之后才能调用。
   - `InitLog` / `InitTracer` / `InitMetrics` 放最前。
   - 用 `init()` 隐式引导时盯紧 import 顺序。
2. **recover 内禁止再 panic**：
   - recover 是最后兜底，里面再崩进程彻底失控。
   - 写法：recover 内只 `fmt.Fprintln(os.Stderr, ...)` 或写本地文件，不依赖任何复杂组件。

> outbox publisher 起不来不算事故——启动顺序错了让进程 crash，才是事故。

cmd/main.go:54–62 tryInitRabbitMQ 的 recover 块；commit 8d0157a 修复点

三档一致性窗口：业务侧约定

档位	窗口上限	业务场景	gomall 落点
强一致	100 ms 内	同一事务可见性、行锁内的预扣	库存预扣（Redis Lua）
近实时一致	1 s 内	用户感知” 操作完成 = 立刻可见”	Outbox publish + ES 索引
最终一致	5 min 内	跨服务对账、数仓同步	死信复盘、cron 巡检

业

务侧承诺（写在对外 SLO 文档里）：

- 用户下单后 $\leq$ **1s** 能在订单列表看到该订单（近实时档）。
- 商品改价后 $\leq$ **1s** 商品详情页显示新价（双删窗口）。
- 库存与订单的对账差异 $\leq$ **5min** 闭合（夜间 cron）。

service/outbox/publisher.go PollInterval=1s 与对账 cron 的窗口设计

事故一：未支付订单取消导致虚高库存

现象（commit 5c9ae27 前）：压测连续下单 + 不支付，15 分钟后**商品可售数变成负数**。

根因链：

1. 旧版 `CancelUnpaidOrder` 把 `product.num` 加回 DB ——但未支付订单**从未真正扣减过** DB 库存 (只占了 Redis reserved)。
2. 第一次取消：DB.num += `n`；Redis.reserved -= `n` ——库存虚高 n。
3. 累计 100 个超时订单 = 库存虚高 100 倍。

修复（PR #58 引入预扣 + Saga）：

- DB.num 在**支付成功**时才递减，未支付订单只动 Redis reserved。
- CancelUnpaidOrder **不再**写 `product.num`，只释放 Redis reserved。
- 库存不变量：`available + reserved + sold = total` 始终成立。

service/order_cancel.go:14–63 注释"不再回写 product.Num"

事故二：搜索索引腐烂 12 小时无人知

现象（commit a43cf61 前）：商品改价后，搜索结果一直显示旧价；商详页正常。

根因链：

1. 旧链路：商品 UPDATE 时**同步**调用 ES 重建索引。
2. ES 集群偶发拒连 → 同步调用失败 → 业务 `rollback`。
3. 业务为了不影响下单，把 ES 写入改成**忽略错误**。
4. ES 失败被吞，索引与 DB 漂移 12 小时无人知。

修复（PR #59 切到 outbox + 消费者）：

- 商品 UPDATE 同事务**只写 outbox_event**，不直接写 ES。
- ES 消费者从 RMQ 拉事件，失败有重试 + 死信。
- 死信触发 alarm，值班同学 5 分钟内看到。

事故三：进程整体 crash 而非降级

现象：RabbitMQ 服务下线维护时，gomall 进程**整体退出**，HTTP 接口全 503。

根因链：

1. `tryInitRabbitMQ()` 的 `recover` 块里调用 `LogrusObj.Warnf`。
2. 初始化序列中 `InitLog()` 反而在它**之后**。
3. `LogrusObj == nil` → recover 内再次 panic → 进程退出。
4. 业务影响：RMQ 是**可选依赖**（延迟关单可降级），但实际把整个进程拖死。

修复（commit 8d0157a / 3cbe3c4）：

- `InitLog()` 上调到 `InitConfig()` 之后第一个位置。
- `recover` 内禁止调用任何依赖全局对象的代码。
- 把 RMQ / ES / Web3 / Milvus 全部走 `tryInit*` 包装，降级而非崩溃。

cmd/main.go:30–47 / stressTest/REPORT.md § 已修：util.LogrusObj

业务侧总结：一致性约束一张表

一致性对	模式	业务码 / 表	gomall 现状
DB ↔ MQ	Transactional Outbox	`outbox_event.status`	pending/sent/dead
DB ↔ Cache	Cache Aside + 延迟双删	`product:detail:{id}`	500ms 延迟双删
缓存击穿	SETNX 单飞回源	`product:lock:{id}`	TTL 3s
跨服务 Saga	协同式 (事件驱动)	见 outbox 路由 key	5 步链路
启动依赖	顺序约束	N/A	InitLog 第 2 位

cmd/main.go / repository/cache/product.go / repository/db/dao/outbox.go

面试 Q&A（1/3）

Q1：为什么 Outbox 比”DB commit 后再 publish” 安全?

A：commit 与 publish 不在一个事务，commit 成功 publish 失败就丢事件。Outbox 把事件写进 DB，由轮询消化，事件丢失等价于 DB 数据丢失。*service/order_cancel.go:30–51*

Q2：outbox 表会不会堆积?

A：会。监控 `status=pending AND next_retry_at <= now()` 的行数即可。横向扩展可用 `SELECT FOR UPDATE SKIP LOCKED`；当前 gomall 单实例 BatchSize=100/1s 还远没到瓶颈。*repository/db/dao/outbox.go:44–51*

Q3：延迟双删的” 延迟” 该取多大?

A：取**大于**最长读 race 窗口。gomall 取 500ms：DB 写常态 5–20ms，留 20× 余量盖 GC / 网络抖动；TTL 10min 兜底。*repository/cache/product.go:64–74*

Q4：协同式 vs 编排式 Saga，怎么选?

A：步骤 ≤ 5、跨团队少 $\rightarrow$ 协同（gomall 选）。步骤多、需可视化、跨团队多 $\rightarrow$ 编排，但要承担” 中心节点” 运维成本。*docs/blog/04-outbox-saga.md § 完整链路*

面试 Q&A（2/3）

Q5：为什么 cache 释放失败不回滚 DB?

A：DB 是业务真相，cache 是性能侧。Cache 失败让库存” 短暂少一点”，最坏拒单不超卖；回滚 DB 反而让” 已关单” 被复活，业务码乱套。*service/order_cancel.go:58–62*

Q6：为什么不用 CDC (Debezium / binlog) 替代 Outbox?

A：CDC 更通用但要拉 binlog 链路、跑 Debezium 集群，运维重；Outbox 是” 应用层 CDC”，简单可控，gomall 量级足够。*service/outbox/publisher.go* 全局

Q7：2PC / XA 为什么互联网业务不用?

A：协调者单点 + 同步阻塞 + 异构资源不支持 (Redis/RMQ/ES 都没 XA)。CAP 三选二里 2PC 强 CP，gomall 选 AP + 业务码不变量。*docs/blog/04-outbox-saga.md § 为什么不用 2PC*

Q8：TCC 与 Outbox 在 gomall 里有什么映射?

A：库存 reserved 字段实际上就是 TCC 的 Try 阶段，但 Confirm/Cancel 不走 TCC 同步协议，改成 outbox 异步事件触发——等价语义、运维低一档。*service/inventory/reservation.go*

面试 Q&A（3/3）

Q9：MaxAttempts=10 怎么算出来的？退避 5min 封顶又是为什么？

A：1+2+4+8+16+32+64+128+256+300=811s≈13.5min 累计窗口，覆盖一次” 中等故障” 够用；封顶 5min 防止退避无限拉长导致积压。*repository/db/dao/outbox.go:62–78*

Q10：Outbox 表会不会成为整库的写入热点？

A：当前峰值 < 100 ev/s，瓶颈在 publish 而非 INSERT；如果突破 `innodb_io_capacity`，按 `aggregate_type` 分表 + 不同 publisher 消费。*repository/db/model/outbox.go idx_outbox_dispatch 索引*

Q11：Event Sourcing 比 Outbox 强在哪？为什么 gomall 不上？

A：ES 把事件流当真相，天然支持回放/审计；代价是读模型必须维护投影 + 快照。gomall 当前规模 ROI 不正，但 outbox_event 表只追加、不删除，已经为未来演进留了门。*repository/db/dao/outbox.go 无 Delete 路径*

Q12：业务侧怎么承诺” 最终一致” 的窗口？

A：分三档：库存预扣 100ms、订单/搜索可见 1s、跨服务对账 5min。对外 SLO 写明上限，超过触发 P2 报警。

代码位置一览

outbox_event 表 *repository/db/model/outbox.go:10–37*

Outbox DAO (Insert / FetchBatch / MarkSent / MarkFailed)

repository/db/dao/outbox.go:13–86

Publisher (Start / loop / drainBatch) *service/outbox/publisher.go:34–86*

初始化 publisher *initialize/outbox.go:12–30*

OrderCreate (业务 + outbox 同事务) *service/order.go:40–102*

CancelUnpaidOrder (Saga 回滚) *service/order_cancel.go:14–63*

缓存双删 / SETNX 回源锁 *repository/cache/product.go:13–74*

ProductUpdate 调用双删 *service/product.go:271–289*

启动顺序修复点 *cmd/main.go:29–47；commit 8d0157a*

配套阅读：deck 04-outbox-saga.tex / deck 03-cache-consistency.tex / docs/blog/04-outbox-saga.md